

CS3213: Foundations of Software Engineering

Test-case Reduction



(Image by Gemini)

Anecdote: Mozilla

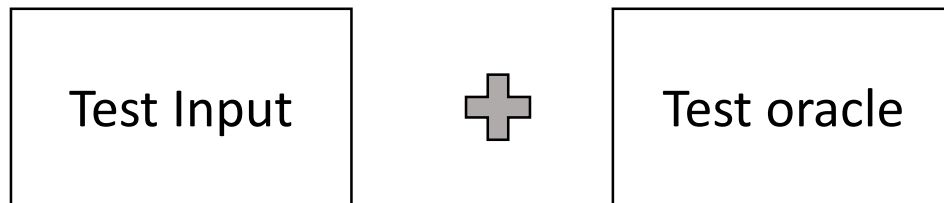
BROWSER BUGS SUCK!
Click here and join the BugAthon!

- In July 1999, Bugzilla, the Mozilla bug database, **listed more than 370 open, unsimplified bug reports**, and the queue was growing
- Seeing that Mozilla engineers “faced imminent doom,” Eric Krock, the Netscape product manager, sent out the Mozilla BugAthon, a call for volunteers to help **simplify bug reports**

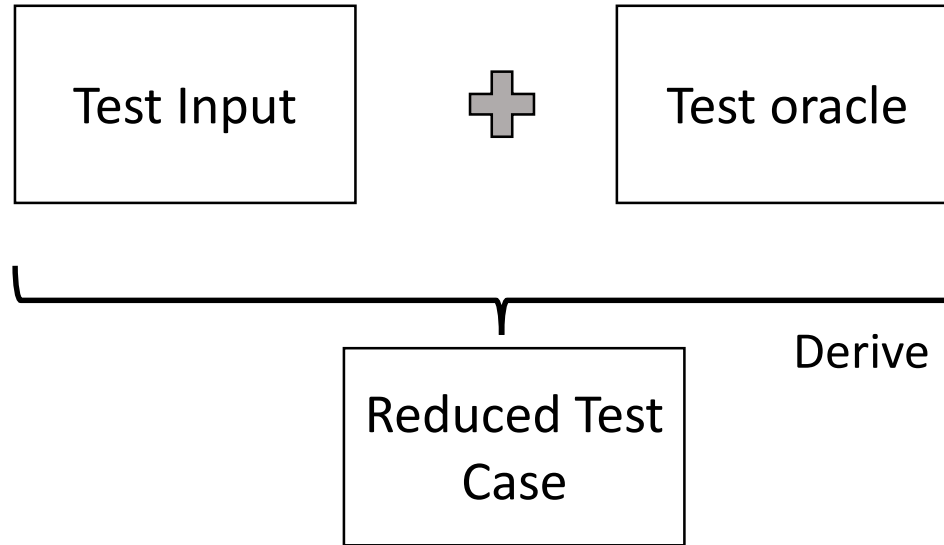
Anecdote: Mozilla

*“Using a text editor (like Notepad on Windows, SimpleText on the Mac, or vi or emacs on UNIX), start **removing HTML markup, CSS rules, and lines of JavaScript** from the page. Start by commenting out parts of the page (using `<!-- -->`) that seem unrelated to the bug. Every few minutes, **check the page to make sure it still reproduces the bug**. Code not required to reproduce the bug can be safely removed.”*

Test Case Reduction



Test Case Reduction



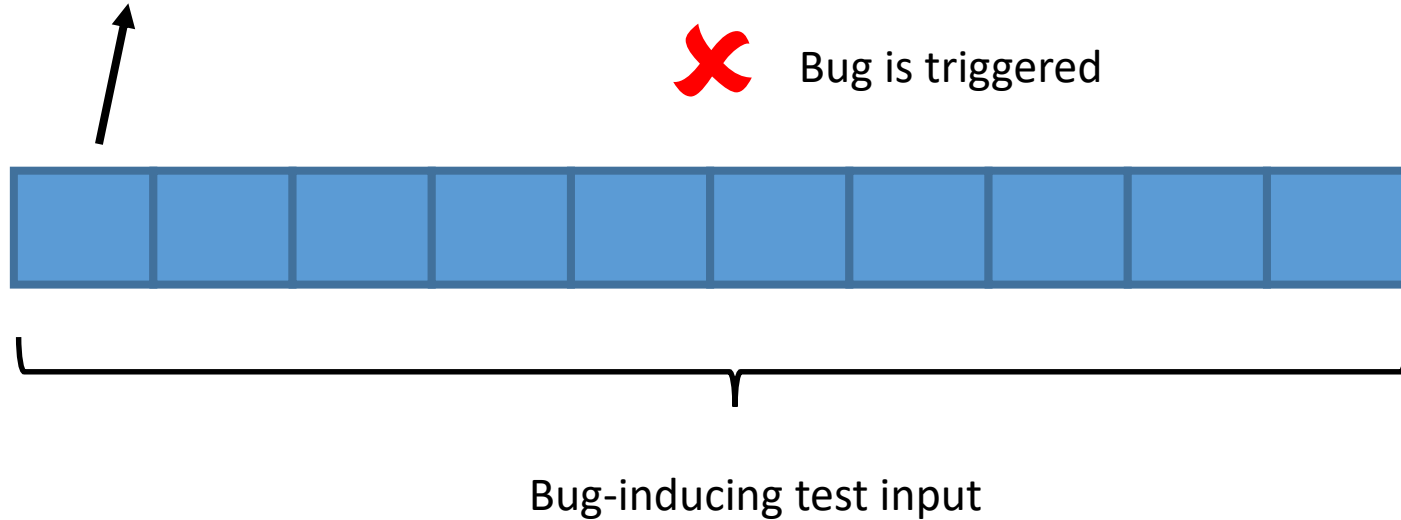


Test Case Reduction

- Test-case reduction tools reduce an input element, apply the (user-supplied) test oracle, and check whether the bug is still being triggered
 - If so, it continues applying another reduction
 - If not, it undoes the change and attempts another reduction

Naïve Approach: Binary Search

Can be a character, line, keyword, ...



Naïve Approach: Binary Search

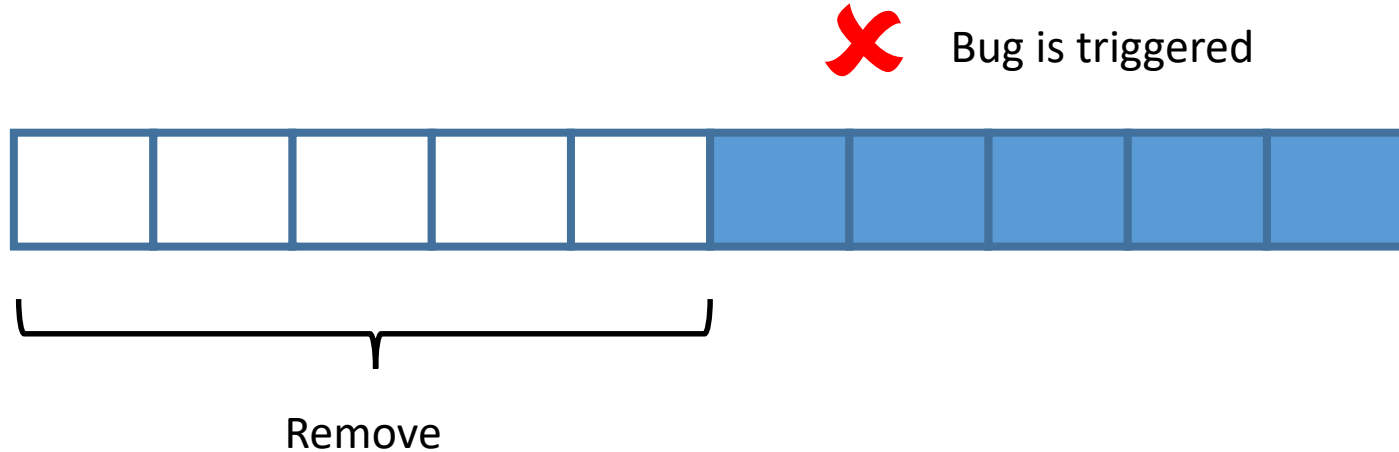


Bug no longer triggers

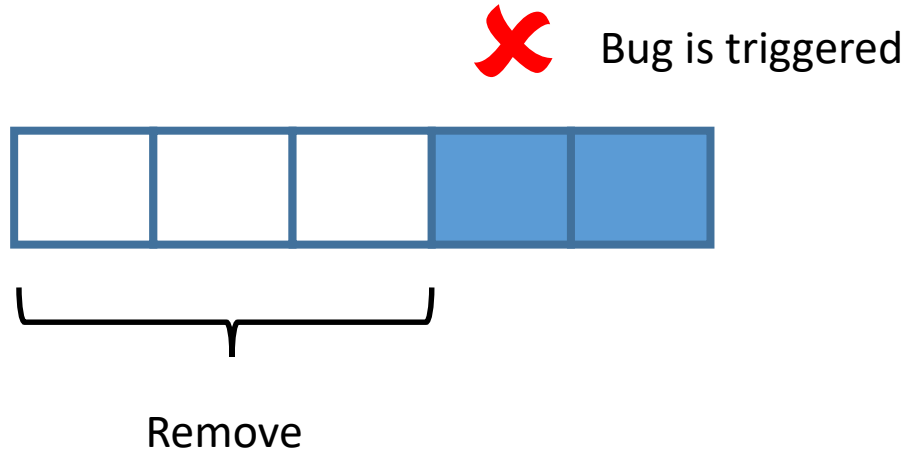


Remove

Naïve Approach: Binary Search



Naïve Approach: Binary Search



Naïve Approach: Binary Search



Bug no longer triggers



Remove

Naïve Approach: Binary Search



Bug no longer triggers



Remove

Naïve Approach: Binary Search

Reduced test case that still triggers the bug



Naïve Approach: Binary Search

- Issue: Case where both halves pass
 - Assume that <SELECT> triggers the bug

<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>



<SELECT NAME="priority" MULTIPLE SIZE=7>





Delta Debugging

- Systematically removes elements from the bug-inducing input
- Behaves like binary search but tries **different combinations of smaller blocks** where binary search fails

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = dmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $dmin(c)$ is

$dmin(c_{\mathbf{x}}) = dmin_2(c_{\mathbf{x}}, 2)$ where

$$dmin_2(c'_{\mathbf{x}}, n) = \begin{cases} dmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ (“reduce to subset”)} \\ dmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ (“reduce to complement”)} \\ dmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ (“increase granularity”)} \\ c'_{\mathbf{x}} & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $dmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

c_x is the failing test case

Minimizing Delta Debugging Algorithm

Let $test$ and c_x be given

The goal is to find c'_x

The minimizing Delta

1. <SELECT NAME="priority" MULTIPLE SIZE=7>

$ddmin(c_x) = ddmin_2(c_x, 2)$ where

$$ddmin_2(c'_x, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_x, \min(|c'_x|, 2n)) & \text{else if } n < |c'_x| \text{ ("increase granularity")} \\ c'_x & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_x - \Delta_i$, $c'_x = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_x|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_x) = \mathbf{x} \wedge n \leq |c'_x|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = dmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $dmin(c_{\mathbf{x}})$ is

$dmin(c_{\mathbf{x}})$

The test function indicates whether the bug is still being triggered (indicated by $\mathbf{x}$)

subset")

$$dmin_2(c'_{\mathbf{x}}, n) = \begin{cases} dmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ dmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $dmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

Finds a smaller (or equal) test case $c_{\mathbf{x}}'$ that still fails

$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

Finding the global minimum would have an **exponential complexity** ($2^{|c_{\mathbf{x}}|}$), which is why Delta Debugging only gives us a local minimum: no subset of the reduced test case the test to fail, which is called **1-minimality**

subset")
complement")

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$.
The goal is to find $c'_{\mathbf{x}} = dmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$.
The minimizing Delta Debugging algorithm $dmin(c_{\mathbf{x}})$ is defined as follows:

We start reducing considering a granularity of $n=2$ (two halves)

$dmin(c_{\mathbf{x}}) = dmin_2(c_{\mathbf{x}}, 2)$ where

$$dmin_2(c'_{\mathbf{x}}, n) = \begin{cases} dmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ dmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ dmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.
The recursion invariant (and thus precondition) for $dmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \mathbf{x}$

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that

The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

We consider the partitions of the input ...

$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2)$ where

$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \mathbf{v}$

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that

The minimizing Delta Debugging algorithm $ddmin(c)$ is

... as well as its complements

$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2)$ where

$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$ $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

If any of the partitions causes the test to fail, we continue reducing this partition

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge$

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2)$ where

$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2)$ where

$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Otherwise, we try
reducing the complement

Delta Debugging

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$.
The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $|c'_{\mathbf{x}}|$ is minimized.
The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

If neither works, we increase the granularity (i.e., creating more partitions)

N = 2

- $\Delta_1 = \langle \text{SELECT NAME} = \text{"priori"}, \Delta_2 = \text{ty} \rangle$ MULTIPLE SIZE=7
- $\nabla_1 = \text{ty} \rangle$ MULTIPLE SIZE=7, $\nabla_2 = \langle \text{SELECT NAME} = \text{"priori}$

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

N = 2

- $\Delta_1 = \langle \text{SELECT NAME} = \text{"priori"} \Delta_2 = \text{ty"} \text{ MULTIPLE SIZE} = 7 \rangle$
- $\nabla_1 = \text{ty"} \text{ MULTIPLE SIZE} = 7 \rangle, \nabla_2 = \langle \text{SELECT NAME} = \text{"priori}$

Minimizing Delta Debugging Algorithm

We cannot reduce to subset

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}, test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i, c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

N = 2

- $\Delta_1 = \langle \text{SELECT NAME} = \text{"priori"}, \Delta_2 = \text{"ty"} \text{ MULTIPLE SIZE} = 7 \rangle$ ✓
- $\nabla_1 = \text{"ty"} \text{ MULTIPLE SIZE} = 7 \rangle, \nabla_2 = \langle \text{SELECT NAME} = \text{"priori}$

Minimizing Delta Debugging Algorithm

We cannot reduce to subset

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}, test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i, c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

N = 2

- $\Delta_1 = \langle \text{SELECT NAME} = \text{"priori"}, \Delta_2 = \text{ty} \rangle$ MULTIPLE SIZE=7
- $\nabla_1 = \text{ty} \rangle$ MULTIPLE SIZE=7, $\nabla_2 = \langle \text{SELECT NAME} = \text{"priori}$



Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

We cannot reduce to complement (and would not need to check as they are the same as other partitions)

N = 2

- $\Delta_1 = \langle \text{SELECT NAME} = \text{"priori"}, \Delta_2 = \text{ty} \rangle$ MULTIPLE SIZE=7
- $\nabla_1 = \text{ty} \rangle$ MULTIPLE SIZE=7, $\nabla_2 = \langle \text{SELECT NAME} = \text{"priori}$

Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Need to increase the granularity with $n = 4$

N = 4

$c_{\mathbf{x}}' = \langle \text{SELECT NATy}'' \text{ MULTIPLE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ME} = \text{"priority"}, \Delta_3 = \text{ty}'' \text{ MULTIP}, \Delta_4 = \text{LE SIZE}=7 \rangle$
- $\nabla_1 = \text{ME} = \text{"priority"} \text{ MULTIPLE SIZE}=7 \rangle, \nabla_2 = \langle \text{SELECT NATy}'' \text{ MULTIPLE SIZE}=7 \rangle, \nabla_3 = \langle \text{SELECT NAME} = \text{"prioriLE SIZE}=7 \rangle, \nabla_4 = \langle \text{SELECT NAME} = \text{"priority"} \text{ MULTIP}$

Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{X}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{X}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{X} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{X} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{X} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 4

$c_{\mathbf{x}}' = \langle \text{SELECT NATy}'' \text{ MULTIPLE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ME} = \text{"priority"}, \Delta_3 = \text{ty}'' \text{ MULTIP}, \Delta_4 = \text{LE SIZE}=7 \rangle$
- $\nabla_1 = \text{ME} = \text{"priority"} \text{ MULTIPLE SIZE}=7 \rangle, \nabla_2 = \langle \text{SELECT NATy}'' \text{ MULTIPLE SIZE}=7 \rangle, \nabla_3 = \langle \text{SELECT NAME} = \text{"prioriLE SIZE}=7 \rangle, \nabla_4 = \langle \text{SELECT NAME} = \text{"priority"} \text{ MULTIP}$

None of the deltas fails the test



Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \times$ hold.

The goal is to find $c_{\mathbf{x}}' = ddmin(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, test(c_{\mathbf{x}}') = \times$, and $c_{\mathbf{x}}'$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c_{\mathbf{x}}', n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \times \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \times \text{ ("reduce to complement")} \\ ddmin_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c_{\mathbf{x}}') = \times \wedge n \leq |c_{\mathbf{x}}'|$.

N = 4

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ME} = \text{"priori"}, \Delta_3 = \text{ty} \rangle$ MULTIP, $\Delta_4 = \text{LE SIZE} = 7 \rangle$
- $\nabla_1 = \text{ME} = \text{"priority"} \rangle$ MULTIPLE SIZE=7, $\nabla_2 = \langle \text{SELECT NATy} \rangle$ MULTIPLE SIZE=7, $\nabla_3 = \langle \text{SELECT NAME} = \text{"priori"} \rangle$ LE SIZE=7, $\nabla_4 = \langle \text{SELECT NAME} = \text{"priority"} \rangle$ MULTIP

Passes since the <SELECT is missing



Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = ddmin(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $test(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c'_{\mathbf{x}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

N = 4

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ME} = \text{"priori"}, \Delta_3 = \text{ty} \rangle$ MULTIP, $\Delta_4 = \text{LE SIZE} = 7 \rangle$
- $\nabla_1 = \text{ME} = \text{"priority"} \text{ MULTIPLE SIZE} = 7 \rangle$, $\nabla_2 = \langle \text{SELECT NATy} \rangle$ MULTIPLE SIZE = 7, $\nabla_3 = \langle \text{SELECT NAME} = \text{"priori"} \text{ LE SIZE} = 7 \rangle$, $\nabla_4 = \langle \text{SELECT NAME} = \text{"priority"} \text{ MULTIP}$

Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(c_{\mathbf{x}}) = \mathbf{x}$.

The goal is to find $c'_{\mathbf{x}} = \text{ddmin}(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} = c_{\mathbf{x}}$, $\text{test}(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c'_{\mathbf{x}}, n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

∇_2 still triggers the bug as it contains $\langle \text{SELECT} \rangle$, which means we can reduce to complement ($n = 3$)



N = 3

$c_{\mathbf{x}}' = \langle \text{SELECT NAty}'' \text{ MULTIPLE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ty}'' \text{ MULTIP}, \Delta_3 = \text{LE SIZE}=7 \rangle$
- $\nabla_1 = \text{ty}'' \text{ MULTIP LE SIZE}=7 \rangle, \nabla_2 = \langle \text{SELECT NALE SIZE}=7 \rangle, \nabla_3 = \langle \text{SELECT NAty}'' \text{ MULTIP}$

Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{X}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{X}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{X} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{X} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{X} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 3

$c_{\mathbf{x}}' = \langle \text{SELECT NATy}'' \text{ MULTIPLE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ty}'' \text{ MULTIP}, \Delta_3 = \text{LE SIZE}=7 \rangle$
- $\nabla_1 = \text{ty}'' \text{ MULTIP LE SIZE}=7 \rangle, \nabla_2 = \langle \text{SELECT NALE SIZE}=7 \rangle, \nabla_3 = \langle \text{SELECT NATy}'' \text{ MULTIP}$

(Still) none of the deltas fails the test



Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{X}$ hold.

The goal is to find $c_{\mathbf{x}}' = ddmin(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, test(c_{\mathbf{x}}') = \mathbf{X}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$ddmin_2(c_{\mathbf{x}}', n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{X} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{X} \text{ ("reduce to complement")} \\ ddmin_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c_{\mathbf{x}}') = \mathbf{X} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 3

$c_{\mathbf{x}}' = \langle \text{SELECT NAty}'' \text{ MULTIPLE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{ty}'' \text{ MULTIP}, \Delta_3 = \text{LE SIZE}=7 \rangle$
- $\nabla_1 = \text{ty}'' \text{ MULTIP LE SIZE}=7 \rangle$, $\nabla_2 = \langle \text{SELECT NALE SIZE}=7 \rangle$ $\nabla_3 = \langle \text{SELECT NAty}'' \text{ MULTIP}$

∇_2 fails still fails, so continue with $n = 2$ and $c_{\mathbf{x}}' = \langle \text{SELECT NALE SIZE}=7 \rangle$



Minimizing Delta Debugging Algorithm

Let $test$ and $c_{\mathbf{x}}$ be given such that $test(\emptyset) = \checkmark \wedge test(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c_{\mathbf{x}}' = ddmin(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}$, $test(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_{\mathbf{x}}) = ddmin_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$ddmin_2(c_{\mathbf{x}}', n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ ddmin_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i$, $c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 2

 $c_{\mathbf{x}}' = \langle \text{SELECT NALE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELECT NA}, \Delta_2 = \text{LE SIZE}=7 \rangle$
- $\nabla_1 = \text{LE SIZE}=7 \rangle, \nabla_2 = \langle \text{SELECT NA}$

Neither halves still fails so
increase granularity with $n = 4$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 4

 $c_{\mathbf{x}}' = \langle \text{SELECT NALE SIZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELE}, \Delta_2 = \text{CT NA}, \Delta_3 = \text{LE SI}, \Delta_4 = \text{ZE}=7 \rangle$
- $\nabla_1 = \text{CT NALE SIZE}=7 \rangle, \nabla_2 = \langle \text{SELELE SIZE}=7 \rangle, \nabla_3 = \langle \text{SELECT NAZE}=7 \rangle, \nabla_4 = \langle \text{SELECT NALE SI}$

Continue with $c_{\mathbf{x}}' = \nabla_3$ and $n=3$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 3

 $c_{\mathbf{x}}' = \langle \text{SELECT NAZE}=7 \rangle$

- $\Delta_1 = \langle \text{SELE}, \Delta_2 = \text{CT NA}, \Delta_3 = \text{ZE}=7 \rangle$
- $\nabla_1 = \text{CT NAZE}=7 \rangle, \nabla_2 = \langle \text{SELEZE}=7 \rangle, \nabla_3 = \langle \text{SELECT NA}$

Increase granularity with $n = 6$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

N = 6

 $c_{\mathbf{x}}' = \langle \text{SELECT NAZE}=7 \rangle$

- $\Delta_1 = \langle \text{SE} \rangle$, $\Delta_2 = \langle \text{LEC} \rangle$, $\Delta_3 = \langle \text{T N} \rangle$, $\Delta_4 = \langle \text{AZ} \rangle$, $\Delta_5 = \langle \text{E} \rangle$, $\Delta_6 = \langle 7 \rangle$
- $\nabla_1 = \langle \text{LECT NAZE}=7 \rangle$, $\nabla_2 = \langle \text{SET NAZE}=7 \rangle$, $\nabla_3 = \langle \text{SELECZE}=7 \rangle$, $\nabla_4 = \langle \text{SELECT NE}=7 \rangle$, $\nabla_5 = \langle \text{SELECT NAZ7} \rangle$, $\nabla_6 = \langle \text{SELECT NAZE} \rangle$

Continue with $c_{\mathbf{x}}' = \nabla_4$ and $n = 5$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \times$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}$, $\text{test}(c_{\mathbf{x}}') = \times$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$
$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \times \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \times \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i$, $c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \times \wedge n \leq |c_{\mathbf{x}}'|$.

$$c_{\mathbf{x}}' = \langle \text{SELECT NE}=7 \rangle$$

N = 5

- $\Delta_1 = \langle \text{SE}, \Delta_2 = \text{LEC}, \Delta_3 = \text{T N}, \Delta_4 = \text{E}, \Delta_5 = 7 \rangle$
- $\nabla_1 = \langle \text{LECT NE}=7 \rangle, \nabla_2 = \langle \text{SET NE}=7 \rangle, \nabla_3 = \langle \text{SELECE}=7 \rangle, \nabla_4 = \langle \text{SELECT NE}=7 \rangle, \nabla_5 = \langle \text{SELECT NE}=7 \rangle$

Continue with $c_{\mathbf{x}}' = \nabla_4$ and $n = 4$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \times$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \times$, and $c_{\mathbf{x}}'$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \times \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \times \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \times \wedge n \leq |c_{\mathbf{x}}'|$.

$$c_{\mathbf{x}}' = \langle \text{SELECT N7} \rangle$$

$$N = 4$$

- $\Delta_1 = \langle \text{SE} \rangle$, $\Delta_2 = \langle \text{LEC} \rangle$, $\Delta_3 = \langle \text{T N} \rangle$, $\Delta_4 = \langle 7 \rangle$
- $\nabla_1 = \langle \text{LECT N7} \rangle$, $\nabla_2 = \langle \text{SET N7} \rangle$, $\nabla_3 = \langle \text{SELEC7} \rangle$, $\nabla_4 = \langle \text{SELECT N} \rangle$

Increase granularity with $n = 8$ ✓

Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = \text{ddmin}(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}$, $\text{test}(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$\text{ddmin}_2(c'_{\mathbf{x}}, n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i$, $c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

$$c_{\mathbf{x}}' = \langle \text{SELECT N7} \rangle$$

N = 8

- $\Delta_1 = \langle S, \Delta_2 = \text{EL}, \Delta_3 = \text{EC}, \Delta_4 = \text{T}, \Delta_5 = , \Delta_6 = \text{N}, \Delta_7 = 7, \Delta_8 = \rangle$
- $\nabla_1 = \text{ELECT N7}, \nabla_2 = \langle \text{SECT N7} \rangle, \nabla_3 = \langle \text{SELT N7} \rangle, \nabla_4 = \langle \text{SELEC N7} \rangle, \nabla_5 = \langle \text{SELECTN7} \rangle, \nabla_6 = \langle \text{SELECT 7} \rangle, \nabla_7 = \langle \text{SELECT N} \rangle, \nabla_8 = \langle \text{SELECT N7} \rangle$

Continue with $c_{\mathbf{x}}' = \nabla_5$ and $n = 7$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

$$c_{\mathbf{x}}' = \langle \text{SELECTN7} \rangle$$

N = 7

- $\Delta_1 = \langle S, \Delta_2 = \text{EL}, \Delta_3 = \text{EC}, \Delta_4 = \text{T}, \Delta_5 = \text{N}, \Delta_6 = 7, \Delta_7 = \rangle$
- $\nabla_1 = \text{ELECTN7}, \nabla_2 = \langle \text{SECTN7} \rangle, \nabla_3 = \langle \text{SELTN7} \rangle, \nabla_4 = \langle \text{SELECN7} \rangle, \nabla_5 = \langle \text{SELECT7} \rangle, \nabla_6 = \langle \text{SELECTN} \rangle, \nabla_7 = \langle \text{SELECTN7} \rangle$

Continue with $c_{\mathbf{x}}' = \nabla_5$ and $n = 6$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c_{\mathbf{x}}' = \text{ddmin}(c_{\mathbf{x}})$ such that $c_{\mathbf{x}}' \subseteq c_{\mathbf{x}}, \text{test}(c_{\mathbf{x}}') = \mathbf{x}$, and $c_{\mathbf{x}}'$ is 1-minimal.

The minimizing Delta Debugging algorithm $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$\text{ddmin}_2(c_{\mathbf{x}}', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_{\mathbf{x}}', \min(|c_{\mathbf{x}}'|, 2n)) & \text{else if } n < |c_{\mathbf{x}}'| \text{ ("increase granularity")} \\ c_{\mathbf{x}}' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_{\mathbf{x}}' - \Delta_i, c_{\mathbf{x}}' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_{\mathbf{x}}'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_{\mathbf{x}}') = \mathbf{x} \wedge n \leq |c_{\mathbf{x}}'|$.

$$c_{\mathbf{x}}' = \langle \text{SELECT7} \rangle$$

N = 6

- $\Delta_1 = \langle S, \Delta_2 = \text{EL}, \Delta_3 = \text{EC}, \Delta_4 = \text{T}, \Delta_5 = 7, \Delta_6 = \rangle$
- $\nabla_1 = \text{ELECT7}, \nabla_2 = \langle \text{SECT7} \rangle, \nabla_3 = \langle \text{SELT7} \rangle, \nabla_4 = \langle \text{SELEC7} \rangle, \nabla_5 = \langle \text{SELECT} \rangle, \nabla_6 = \langle \text{SELECT7} \rangle$



Minimizing Delta Debugging Algorithm

Let test and $c_{\mathbf{x}}$ be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_{\mathbf{x}}) = \mathbf{x}$ hold.

The goal is to find $c'_{\mathbf{x}} = \text{ddmin}(c_{\mathbf{x}})$ such that $c'_{\mathbf{x}} \subseteq c_{\mathbf{x}}, \text{test}(c'_{\mathbf{x}}) = \mathbf{x}$, and $c'_{\mathbf{x}}$ is 1-minimal.

The *minimizing Delta Debugging algorithm* $\text{ddmin}(c)$ is

$$\text{ddmin}(c_{\mathbf{x}}) = \text{ddmin}_2(c_{\mathbf{x}}, 2) \quad \text{where}$$

$$\text{ddmin}_2(c'_{\mathbf{x}}, n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \mathbf{x} \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \mathbf{x} \text{ ("reduce to complement")} \\ \text{ddmin}_2(c'_{\mathbf{x}}, \min(|c'_{\mathbf{x}}|, 2n)) & \text{else if } n < |c'_{\mathbf{x}}| \text{ ("increase granularity")} \\ c'_{\mathbf{x}} & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c'_{\mathbf{x}} - \Delta_i, c'_{\mathbf{x}} = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\mathbf{x}}|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c'_{\mathbf{x}}) = \mathbf{x} \wedge n \leq |c'_{\mathbf{x}}|$.

Continue with $c_{\mathbf{x}}' = \nabla_5$ and $n = 5$

$$c_X' = \langle \text{SELECT} \rangle$$

N = 5

- $\Delta_1 = \langle S, \Delta_2 = \text{EL}, \Delta_3 = \text{EC}, \Delta_4 = \text{T}, \Delta_5 = \rangle$
- $\nabla_1 = \text{ELECT}, \nabla_2 = \langle \text{SECT} \rangle, \nabla_3 = \langle \text{SELT} \rangle, \nabla_4 = \langle \text{SELEC} \rangle, \nabla_6 = \langle \text{SELECT} \rangle$

Minimizing Delta Debugging Algorithm

Continue with n = 8



Let test and c_X be given such that $\text{test}(\emptyset) = \checkmark \wedge \text{test}(c_X) = \times$ hold.

The goal is to find $c_X' = \text{ddmin}(c_X)$ such that $c_X' \subseteq c_X$, $\text{test}(c_X') = \times$, and c_X' is 1-minimal.

The *minimizing Delta Debugging algorithm* $\text{ddmin}(c)$ is

$$\text{ddmin}(c_X) = \text{ddmin}_2(c_X, 2) \quad \text{where}$$

$$\text{ddmin}_2(c_X', n) = \begin{cases} \text{ddmin}_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\Delta_i) = \times \text{ ("reduce to subset")} \\ \text{ddmin}_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot \text{test}(\nabla_i) = \times \text{ ("reduce to complement")} \\ \text{ddmin}_2(c_X', \min(|c_X'|, 2n)) & \text{else if } n < |c_X'| \text{ ("increase granularity")} \\ c_X' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_X' - \Delta_i$, $c_X' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_X'|/n$ holds.

The recursion invariant (and thus precondition) for ddmin_2 is $\text{test}(c_X') = \times \wedge n \leq |c_X'|$.

$$c_X' = \langle \text{SELECT} \rangle$$

N = 8

- $\Delta_1 = <, \Delta_2 = S, \Delta_3 = E, \Delta_4 = L, \Delta_5 = E, \Delta_6 = C, \Delta_7 = T, \Delta_8 = >$
- $\nabla_1 = \text{SELECT}, \nabla_2 = \langle \text{ELECT} \rangle, \nabla_3 = \langle \text{SLECT} \rangle, \nabla_4 = \langle \text{SEECT} \rangle, \nabla_5 = \langle \text{SELCT} \rangle, \nabla_6 = \langle \text{SELET} \rangle, \nabla_7 = \langle \text{SELEC} \rangle, \nabla_8 = \langle \text{SELECT} \rangle$

No other steps are applicable, so $\langle \text{SELECT} \rangle$ is the 1-minimal bug-inducing test case



Minimizing Delta

Let $test$ and c_X be

The goal is to find $c_X' = ddmin(c_X)$ such that $c_X' \subseteq c_X$, $test(c_X') = \mathbf{X}$, and c_X' is 1-minimal.

The minimizing Delta Debugging algorithm $ddmin(c)$ is

$$ddmin(c_X) = ddmin_2(c_X, 2) \quad \text{where}$$

$$ddmin_2(c_X', n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \dots, n\} \cdot test(\Delta_i) = \mathbf{X} \text{ ("reduce to subset")} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \dots, n\} \cdot test(\nabla_i) = \mathbf{X} \text{ ("reduce to complement")} \\ ddmin_2(c_X', \min(|c_X'|, 2n)) & \text{else if } n < |c_X'| \text{ ("increase granularity")} \\ c_X' & \text{otherwise ("done").} \end{cases}$$

where $\nabla_i = c_X' - \Delta_i$, $c_X' = \Delta_1 \cup \Delta_2 \cup \dots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c_X'|/n$ holds.

The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c_X') = \mathbf{X} \wedge n \leq |c_X'|$.

Small Scope Hypothesis

- *Small scope hypothesis*: “A high proportion of errors can be found by testing a program for all test inputs within **some small scope**”
- That is, most bugs can be found with small/simple inputs and we can potentially *reduce* the bug-inducing input

Evaluating the “Small Scope Hypothesis”

Alexandr Andoni

Dumitru Daniliuc

Sarfraz Khurshid

Darko Marinov

MIT Laboratory for Computer Science

200 Technology Square

Cambridge, MA 02139

{andoni, dumi, khurshid, marinov}@lcs.mit.edu

Abstract

The “small scope hypothesis” argues that a high proportion of bugs can be found by testing the program for all test inputs within some small scope. In object-oriented programs, a test input is constructed from objects of different classes; a test input is within a scope of s if at most s objects of any given class appear in it. If the hypothesis holds, it follows that it is more effective to do systematic testing within a small scope than to generate fewer test inputs of a larger scope.

This paper evaluates the hypothesis for several implementations of data structures, including some from the Java Collections Framework. We measure how statement coverage, branch coverage, and rate of mutant killing vary with scope. For systematic input generation and correctness checking of Java programs, we use the Korat framework. This paper also presents the Ferastrau framework that we have developed for mutation testing of Java programs. The experimental results show that exhaustive testing within small scopes can achieve complete coverage and kill most of the mutants, even for intricate methods that manipulate complex data structures. The results also show that Korat can be used effectively to generate inputs and check correctness for these scopes.

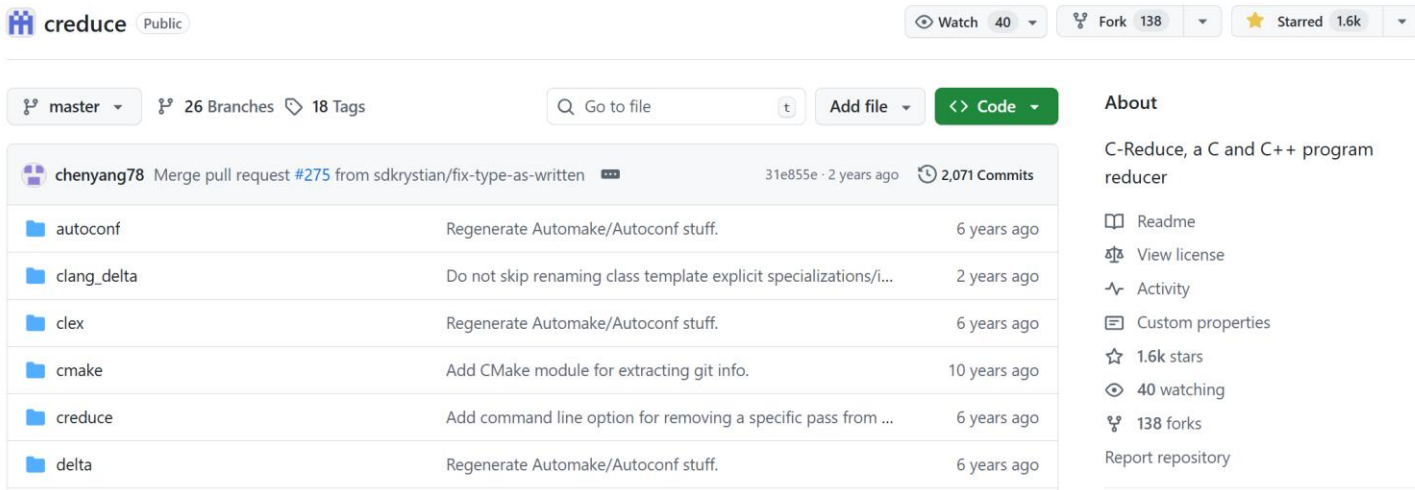
language [12] to develop software models and exhaustively check them for small scopes with the Alloy Analyzer. These case studies showed that the hypothesis holds for those software models, but they did not consider the actual implementations.

This paper evaluates the “small scope hypothesis” for several benchmark programs, including some data structure implementations from the Java Collections Framework [29]. Evaluating the hypothesis requires determining the scope up to which each program should be checked, i.e., the sufficient scope that gives significant confidence that the program has no bugs. We use code coverage and mutation testing criteria to determine the sufficient scope.

Code coverage is a common criterion for assessing the quality of a set of test inputs [5]. Measuring code coverage involves executing the program on each input and recording parts of the program (e.g., statements, branches, paths) that get executed. Statement (branch) coverage is then the ratio of the number of executed statements (branches) to the number of total statements (branches) in the program; complete coverage is the ratio of 100%.

Mutation testing is another criterion for assessing the quality of a set of test inputs [11, 26]. Mutation testing proceeds in two steps. In the first step, several mutants are generated from the original (correct) program, by performing one or more syntactic modifications. These modifications are specified by *mutation operators*, e.g., replacing a vari-

C-Reduce



The screenshot shows the GitHub repository for C-Reduce. At the top, the repository name "creduce" is displayed with a "Public" label. To the right, there are buttons for "Watch" (40), "Fork" (138), and "Starred" (1.6k). Below this, the repository is shown to be on the "master" branch, with 26 branches and 18 tags. A search bar and "Add file" button are present. The main content area lists recent merge pull requests from user "chenyang78", including a request #275 from "sdkrystian/fix-type-as-written". Below this is a table of repository files and folders, each with a description and the time since the last commit. On the right, the "About" section describes C-Reduce as a C and C++ program reducer, and lists repository statistics: 1.6k stars, 40 watching, and 138 forks.

File/Folder	Description	Last Commit
autoconf	Regenerate Automake/Autoconf stuff.	6 years ago
clang_delta	Do not skip renaming class template explicit specializations/i...	2 years ago
cllex	Regenerate Automake/Autoconf stuff.	6 years ago
cmake	Add CMake module for extracting git info.	10 years ago
creduce	Add command line option for removing a specific pass from ...	6 years ago
delta	Regenerate Automake/Autoconf stuff.	6 years ago

About
C-Reduce, a C and C++ program reducer

- Readme
- View license
- Activity
- Custom properties

☆ 1.6k stars
👁 40 watching
🍴 138 forks
Report repository

<https://github.com/csmith-project/creduce>



Reducer Characteristics

- Advantages
 - Easier to debug and analyze the reason of the bug
 - Might help identifying duplicate bug-inducing test cases
- Challenges
 - Interestingness test can be complex
 - Delta Debugging might be inefficient for highly structured languages (additional reduction approaches such as hierarchical delta debugging have been proposed)



Summary and Takeaways

- Reducer: operates based on an input and interestingness test
- Binary search is insufficient
- The delta debugging algorithm (ddmin) is a systematic approach that guarantees 1-minimality
- The small-scope hypothesis suggests that most bugs can be found by test inputs within some small scope